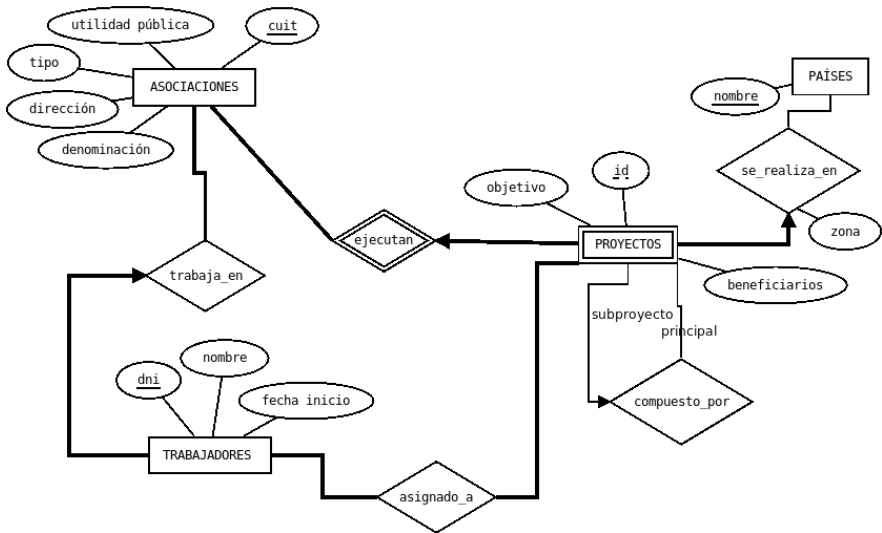


1) (4 puntos) Dado el siguiente DER:



El diagrama corresponde a la base de datos del subsistema de gestión de proyectos de ONGs. Los proyectos dependen enteramente de las asociaciones que los ejecutan, identificándose por un número correlativo dentro de éstas. Un proyecto puede estar compuesto por otros proyectos y se debe especificar en qué país se lleva a cabo, especificando además en qué zona del país se realiza. Se registran los datos de los trabajadores que trabajan en las asociaciones y a qué proyectos son asignados.

Se pide:

- a) Escriba las **sentencias SQL** para crear el **esquema relacional** correspondiente utilizando las **cláusulas** y **parámetros** que considere necesarios para responder de la manera más adecuada a las siguientes consignas:
  - i. Escribir como un comentario fecha y hora en que está realizando la tarea;

-- Tarea realizada el 29/09/2022 a partir de las 20,15hs
  - ii. Crear la base de datos *Primer parcial T1*;

CREATE DATABASE Primer\_parcial\_T1;
  - iii. Crear las tablas especificando las restricciones de clave (primarias, candidatas), clave foránea (y manejo de integridad referencial), dominio, campos no nulos, y demás opciones que correspondan para capturar la semántica del DER;

/\* Lo correcto en este punto sería tener en cuenta también cómo prevenir la creación de claves foráneas que referencien a tablas que aún no se crearon \*/

```
CREATE TABLE ASOCIACIONES (  
    cuit BIGINT,  
    denominacion VARCHAR(25) NOT NULL,  
    utilidad_publica VARCHAR(25),  
    tipo CHAR(20) NOT NULL,  
    direccion TEXT,  
    PRIMARY KEY (cuit)  
);  
  
CREATE TABLE PAISES (  
    nombre CHAR(20),  
    PRIMARY KEY (nombre)  
);  
  
CREATE TABLE PROYECTOS (  
    cuit BIGINT,  
    id SMALLINT,  
    objetivo VARCHAR(100) NOT NULL,  
    beneficiarios VARCHAR(50),  
    pais CHAR(20) NOT NULL,  
    zona VARCHAR(20),  
    p_ppal_cuit BIGINT,  
    p_ppal_id SMALLINT,  
    PRIMARY KEY (cuit, id),  
    FOREIGN KEY (cuit) REFERENCES ASOCIACIONES(cuit)  
        ON DELETE CASCADE ON UPDATE CASCADE,  
    FOREIGN KEY (pais) REFERENCES PAISES(nombre)  
        ON DELETE NO ACTION ON UPDATE CASCADE  
);  
  
ALTER TABLE PROYECTOS  
    ADD FOREIGN KEY(p_ppal_cuit, p_ppal_id) REFERENCES PROYECTOS(cuit, id)  
        ON UPDATE CASCADE ON DELETE NO ACTION
```

```

;
CREATE TABLE TRABAJADORES (
    dni INT(8),
    nombre VARCHAR(50) NOT NULL,
    fecha_inicio DATE,
    asociacion BIGINT NOT NULL,
    PRIMARY KEY (dni),
    FOREIGN KEY (asociacion) REFERENCES ASOCIACIONES(cuit)
        ON DELETE NO ACTION ON UPDATE NO ACTION
);
CREATE TABLE asignado_a (
    trabajador INT(8),
    proyecto_cuit BIGINT,
    proyecto_id SMALLINT,
    PRIMARY KEY (trabajador, proyecto_cuit, proyecto_id),
    FOREIGN KEY (trabajador) REFERENCES TRABAJADORES(dni)
        ON UPDATE CASCADE ON DELETE NO ACTION,
    FOREIGN KEY (proyecto_cuit, proyecto_id) REFERENCES PROYECTOS(cuit, id)
        ON UPDATE CASCADE ON DELETE CASCADE
);

```

iv. Crear un índice que permita optimizar búsquedas por el nombre de los trabajadores;

```
CREATE INDEX por_nombre ON TRABAJADORES (nombre);
```

v. Se decide que cada proyecto debe tener un trabajador como coordinador y debe registrarse además la fecha desde la cual coordina el proyecto. Haga las modificaciones necesarias de acuerdo a lo ya desarrollado en el ítem iii para reflejar esta nueva situación;

```
ALTER TABLE PROYECTOS
    ADD COLUMN coordinador INT NOT NULL,
    ADD COLUMN coordina_desde DATE NOT NULL,
    ADD FOREIGN KEY (coordinador) REFERENCES TRABAJADORES(dni)
        ON UPDATE CASCADE ON DELETE NO ACTION
;

```

b) Realizar las siguientes acciones de LMD, **comentando** previamente si es necesario tener en cuenta alguna **consideración** o realizar **acciones adicionales** (escriba las sentencias SQL que las ejecutan):

i. Registrar un nuevo proyecto con los siguientes datos: **id:** 117, **objetivo:** ‘Capacitar en BBDD a agentes públicos’, **beneficiarios:** ‘Agentes de la administración pública nacional’; la asociación ejecutora del proyecto es ‘Capacitaciones Ad-Hoc’ y el coordinador ‘Gustavo Gómez’;

```

/* Tener en cuenta que previamente deben estar cargados los registros de la asociación y del trabajador. También que los tipos, longitudes, y obligatoriedad de datos definidos en los campos de tabla permitan cargar los valores pedidos */
INSERT INTO PROYECTOS(cuit, id, objetivo, beneficiarios, pais, coordinador, coordina_desde)
SELECT a.cuit, 117, 'Capacitar en BBDD a agentes públicos', 'Agentes de la administración pública nacional',
    'ARGENTINA', t.dni, curdate()
FROM ASOCIACIONES a, TRABAJADORES t
WHERE a.denominacion LIKE 'Capacitaciones Ad-Hoc' AND t.nombre LIKE 'Gustavo Gómez';

```

ii. El trabajador Gustavo Gómez se jubila, por lo que todos los proyectos que estaban bajo su coordinación pasan a estar a cargo de Martín Peralta;

```

/* Previamente debe estar cargado el registro del nuevo coordinador */
/* Se pasan las coordinaciones de los proyectos a cargo de Gustavo Gómez a Martín Peralta */
UPDATE PROYECTOS p, TRABAJADORES t
SET p.coordinador = t.dni, coordina_desde = curdate()
WHERE t.nombre LIKE 'Martin Peralta' AND p.coordinador = (
    SELECT dni
    FROM TRABAJADORES
    WHERE nombre LIKE 'Gustavo Gómez'
);
/* Se refleja la jubilación de Gustavo Gómez */
DELETE FROM TRABAJADORES
WHERE nombre LIKE 'Gustavo Gómez';

```

iii. Eliminar de la base de datos los proyectos realizados en Costa Rica;

```
DELETE FROM PROYECTOS
```

WHERE pais LIKE 'COSTA RICA';

2) (4 puntos) Dado el escenario siguiente resolver los ejercicios mediante consultas SQL:

**Alumnos**(numa: integer, nombrea: string, carrera: string, nivel: string, edad: integer)

**Cursos**(nombre: string, horarios: string, aula: string, idp: integer)

CF: idp → Profesor(idp)

**Matriculado**(numa: integer, nombrec: string, nota: real)

CF: numa → Alumnos(numa); nombrec → Cursos(nombre)

**Profesores**(idp: integer, nombrep: string, iddep: integer)

/\* Hay varias alternativas para cada consulta. Destacar si se usa algo de lo aprendido (p.e. CTE, TDL) \*/

- a) Averiguar la edad del alumno de más edad en la carrera de Historia y que además está matriculado en algún curso dictado por la profesora *Amalia Miranda*.

```
SELECT a.edad
FROM Alumnos a, Matriculado m, Cursos c,
  (SELECT MAX(edad) AS mayor
   FROM Alumnos
   WHERE carrera LIKE 'Historia') AS Tedad,
WHERE a.edad = Tedad.mayor AND a.numa = m.numa AND m.nombrec = c.nombre AND c.idp = (
  SELECT idp
  FROM Profesores
  WHERE nombrep LIKE 'Amalia Miranda')
;
```

- b) Averiguar el nombre de los profesores que dan clase en todas las aulas en las que se dicta algún curso.

```
SELECT p.nombrep
FROM Profesores p
WHERE NOT EXISTS (
  SELECT *
  FROM Cursos c
  WHERE NOT EXISTS (
    SELECT *
    FROM Cursos c1
    WHERE p.idp = c1.idp AND c.aula = c1.aula
  ));
```

- c) Crear una vista que muestre, para cada nivel (excepto el “inicial”), el nivel y las edades promedio, mínima y máxima de los alumnos de ese nivel.

```
CREATE VIEW edades AS
SELECT a.nivel, AVG(), MIN(), MAX()
FROM Alumnos a
WHERE a.nivel NOT LIKE 'inicial'
GROUP BY a.nivel
;
```